Exercise 01:

Gantt Chart

In this exercise, we will explain how to specify the followings:

1. Total duration of a project to be done.
2. Total number of tasks that has to be done.
3. The dependencies between the different tasks.
4. The minimum number of employees that has work on this project based on the number of employees working on each task:
    a. Minimum number of Software Engineers (SEs)
    b. Minimum number of Programmers (Prgs)

Part A: Project Design and Information

Given the following project:

Task #	Task	Duration (Weeks)	Dependency	Number of Software Engineers	Number of Programmers
A	Problem Analysis	2	-	2	-
B	Requirements Analysis	3	A	1	4
C	Logical Design	4	B	1	2
D	Decision Analysis	2	B	2	3
E	Physical Design	5	C,D	2	6
F	Construction and Testing	4	D,E	1	5
G	Implementation and Delivery	6	E	1	4

In the above table, we can check the below information:

1. **Task**: we have here seven different tasks. We may have different number of tasks in different projects; so this is changeable.
2. **Duration**: Each task has a specific duration (in weeks). Some tasks may be measured in days or even months. This task has been already calculated by the project manager using the equation below:

$$Duration = \frac{1 * OD + 4 * ED + 1 * PD}{6}$$

OD: Optimistic duration; ED: Expected Duration; PD: Pessimistic Duration

3. **Dependency**: we specify whether a task is dependent on some other tasks in order to know when to start the current task.

4. **Number of software engineers**: we specify the number of engineers working on each task.
5. **Number of programmers**: we specify the number of programmers working on each task.

Part B: Gantt Chart

Now, we need to build the Gantt chart in order to specify the:

1. Total project duration
2. Minimum number of SEs
3. Minimum number of Progs

	Week																			
Task	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**	**18**	**19**	**20**	**21**
A	20	20																			
B			14	14	14																
C						12	12	12	12												
D						23	23														
E										26	26	26	26	26							
F															15	15	15	15			
G															14	14	14	14	14	14	
# of SEs	2	2	1	1	1	3	3	1	1	2	2	2	2	2	2	2	2	2	1	1	
# of Progs	0	0	4	4	4	5	5	2	2	6	6	6	6	6	9	9	9	9	4	4	

Explanation:

1. We will show that table as duration in weeks; for each task, we specify the number of software engineers and programmers.
2. We need to start a task after the finish of its dependencies.
3. In the table, # of SEs and # of Progs are calculated for each week.
4. Finally, the maximum number of SEs and the maximum number of Progs are the minimum number of employees needed in the project.
5. As a result, and based on the table, we can see the followings:
    a. Total duration: 20 weeks
    b. Minimum number of SEs (needed): 3
    c. Minimum number of Progs (needed): 9

Debate:

Can we make a performance for this project? The answer depends on whether we can expand the duration of the project in order to decrease the number of software engineers and programmers needed; or vice versa.

Exercise 02:

Pert Chart

In this exercise, we will explain how to specify the **Critical Path** of the project.

Part A: Project Design and Information

Based on the project table given in the previous lecture:

Task #	Task	Duration (Weeks)	Dependency	Number of Software Engineers	Number of Programmers
A	Problem Analysis	2	-	2	-
B	Requirements Analysis	3	A	1	4
C	Logical Design	4	B	1	2
D	Decision Analysis	2	B	2	3
E	Physical Design	5	C,D	2	6
F	Construction and Testing	4	D,E	1	5
G	Implementation and Delivery	6	E	1	4

We need to specify the critical path of the project after specifying the critical tasks in this project based on the following data:

Critical Path Definition: the sequence of dependent tasks that determines the earliest possible completion date of the project. This means that we need to specify the tasks that have to start on time and finish on time.

In order to determine this path, we have to determine the following terminologies:

Early Start (ES): The earliest time a task can possibly begin based on all its predecessors and successors.

Early Finish (EF): The earliest time a task can possibly finish based on all its predecessors and successors.

Late Start (LS): The latest time a task can possibly start without affecting the whole planned project finish time.

Late Finish (LF): The latest time a task can possibly finish without affecting the project finish time.

Duration (D): the amount of time that takes a task to get finalized.

Slack Time (ST): the amount of time delay that can be tolerated between the starting time and the finishing time of a specific task.

How to specify the critical task?

A critical task is that task that having zero value as slack time.

Part B: Task Representation

Based on the terminologies shown in Part A, we will be representing the Task with its all data as show in the table below:

Early Start	**Task Number**	**Early Finish**
Slack Time	**Critical? Yes or NO**	
Late Start	**Duration**	**Late Finish**

Example:

0	**Task A**	2
0	Critical? Yes	
0	2	2

5	**Task D**	7
2	Critical? No	
7	2	9

These values have been indicated based on the following equations:

$$\textbf{\textit{Early Start}} = \textbf{\textit{Early Finish}} \text{ (Of the predecessors maximum Early Finish)}$$

$$\textbf{\textit{Early Finish}} = \textbf{\textit{Early Start}} + \textbf{\textit{Duration}} \text{ (Of the same task)}$$

$$\textbf{\textit{Late Finish}} = \textbf{\textit{Late Start}} \text{ (Of the successors minimum Late Start)}$$

$$\textbf{\textit{Late Start}} = \textbf{\textit{Late Finish}} - \textbf{\textit{Duration}} \text{ (Of the same task)}$$

$$\textbf{\textit{Slack Time}} = \textbf{\textit{Late Start}} - \textbf{\textit{Early Start}} \text{ (Of the same task)}$$

Explanation:

1. The Project has to start with one Node (Start) and finish with on Node (End).

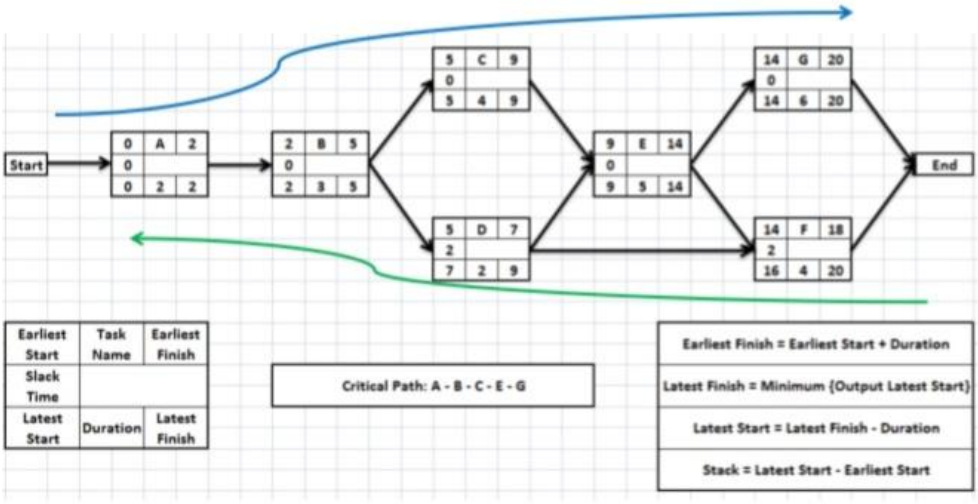
2. Calculating all the Early Start and Early Finish (going from left to right- blue arrow) of the figure shown in the next page.
   a. Early Start is calculated based on all the predecessors' tasks by taking the highest value of predecessors' Early Finish.
   Example: Task E depends on Tasks C and D. This means that we have to take the highest value of Early Finish between Tasks C and D as the Early Start of Task E.
   $$EF\ (C) = 9\ and\ EF\ (D) = 7\ \Rightarrow\ ES\ (E) = 9$$
   Note: Task A has no predecessors so $ES\ (A) = 0$.

   b. Early Finish is calculated as Early Start + Duration of the same task.
   Example: Task A starts at Early Start 0 and has duration of 2 weeks; then $EF\ (A) = 2$.

3. Calculating all the Late Finish and Late Start (going from right to left –green arrow) of the figure shown in the next page.
   a. Late Finish is calculated based on all the successors' tasks by taking the lowest value of successors' Late Start.
   Example: Task B is the dependency task for Tasks C and D. This means that we have to take the lowest value of Late Start between Tasks C and D as the Late Finish of Task B.
   $$LS\ (C) = 5\ and\ LS\ (D) = 7\ \Rightarrow\ LF\ (B) = 5$$
   Note: Tasks F and G have no successors. So, the Late Finish for both of them is the maximum Early Finish between them.
   $$EF\ (F) = 18\ and\ EF\ (G) = 20\ \Rightarrow\ LF\ (F) = LF\ (G) = 20$$

   b. Late Start is calculated as Late Finish - Duration of the same task.
   Example: Task A finishes at Late Finish 2 and has duration of 2 weeks; then $LS\ (A) = 0$.

4. Calculating the Slack Time (ST) for each task to check whether this task is critical or not. The task is critical if its slack time value is 0.
   Example: $ST\ (A) = LS\ (A) - ES\ (A) = 2 - 2 = 0$ and $ST\ (D) = LS\ (D) - ES\ (D) = 7 - 5 = 2$. This means that Task A is critical; however, Task D is not critical.



Remarks:

1. We may have one or more tasks with no predecessors. This means that all these tasks have to start at Node Start.
2. We may have one or more tasks with no successors. This means that all these tasks have to finish at Node Finish.
3. No negative values in the calculation including slack time that can never be a negative value.
4. The critical path is a chain from Start node to End node. This means that we can never have a corrupted path between the different tasks.
   a. As we can see in the figure above, the critical path starts with Task A at start node and finishes with Task G at end node.
   b. The critical path: A, B, C, E, and G is a chain.

Exercise 03:

Medical Clinic System

Medical clinic system is an application used by most of the doctors to keep track of all the patient information including appointment, medical records, and medication payments. This system contains the following information:

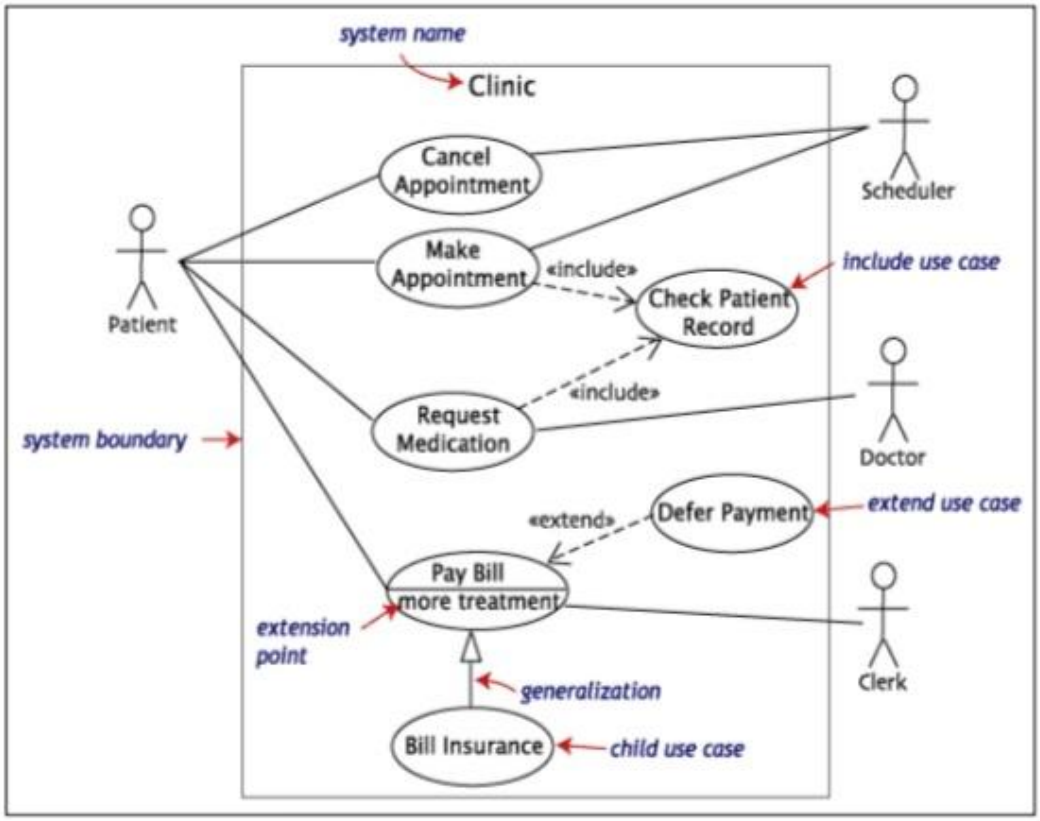
1. The patient can:
   a. Make an appointment,
   b. Cancel an appointment,
   c. Pay the bill either cash money or through an insurance company. However, the bill can be paid through deferred payments (deferred means many payments of smaller amount of money).
2. The scheduler, as well, can:
   a. Make the appointment for the patient.
   b. Cancel the appointment for the patient
3. Any appointment taken needs to check the patient record.
4. The doctor can request medication for the patient.
5. Any medical request needs to check the patient record.
6. The clerk takes the payments from the patient.

Questions:

A. Draw the use-case model diagram:
   a. Show the interaction between actors and each of the use-cases in the system
   b. Show the dependencies among use cases (initiates, includes, extends, or depends on).
B. Create the use-case narratives (check the form on the next page) for "Pay Bill" to document the business requirements.

Solution:

Part A: Use Case Diagram



Part B: Use-Case Narrative

Use-Case Name:	Pay Bill	
Use-Case ID:	MCS - 006	
Priority:	High	
Primary Actor:	Patient	
Description:	This use case describes the event that the patient is paying a bill after visiting the doctor for an appointment and requested for a medication and treatment	
Precondition:	The patient should have made an appointment and visited the doctor	
Typical Course of Events:	**Actor Action**	**System Response**
	1. The patient provides his appointment information and medication record. 5. The patient will choose how to pay (cash, insurance).	2. The system verifies the information given by the patient. 3. If this is the first visit of the patient, the system will add the new information 4. Once the information is verified, the payment bill is generated.
Conclusion:	The use case concludes when the patient pays for the appointment and finally receives a signed receipt.	

Exercise 04:

1. ATM System:
A bank provides an Automated Teller Machine (ATM) system that allows customers to perform various banking operations. A customer must first authenticate using a card and PIN before accessing any services. Once authenticated, the customer can check their account balance, withdraw cash, deposit funds, and transfer money between accounts.

The ATM communicates with the bank system to verify credentials and process transactions. If authentication fails, access is denied. A technician is responsible for maintaining and repairing the ATM, including diagnosing faults and restoring normal operation.

Task:
Construct a Use Case Diagram that identifies all actors, use cases, and relationships. Clearly model authentication, include relationships between use cases where appropriate, and represent interactions with the bank system.

2. Website System:
A content-based website allows users to interact with various types of digital content. A site user can browse documents, search for specific documents, view upcoming events, preview documents, and download them.

A webmaster manages the system by uploading documents, posting new events, managing user accounts, and organizing content into folders. Some actions depend on others, such as previewing before downloading or managing folders before uploading content.

Task:
Draw a Use Case Diagram showing all actors, use cases, and relationships. Include dependencies between actions and distinguish between user and administrator functionalities.

3. Library Management System:
A library system serves students and staff members who want to access library resources. Users must first authenticate before performing operations. They can search for books, reserve books, borrow and return books, renew borrowed books, and pay fines for overdue items.

A librarian manages the system by registering new users, issuing library cards, updating book records, and handling exceptional cases such as invalid renewals. The system also interacts with a library database that stores all records.

Task:
Create a Use Case Diagram identifying all human and system actors. Include all major use

cases and clearly show relationships such as includes, extends, and generalizations where applicable.

4. Online Shopping System:
An online shopping platform allows customers to browse products, view item details, add items to a cart, and proceed to checkout. Customers can either register as new users or log in as registered users. During checkout, the system calculates totals and processes payments.

External payment services such as PayPal and credit card processors are involved in completing transactions. The system must validate payment details before confirming the purchase.

Task:
Develop a Use Case Diagram that includes all types of users and external systems. Show relationships between use cases such as checkout including payment processing, and distinguish between registered and new users.

5. Hospital Management System:
A hospital management system is used primarily by a receptionist who handles patient interactions. The receptionist can schedule patient appointments, register new patients, admit patients to the hospital, and manage patient records.

Some actions depend on others, such as registering a patient before scheduling an appointment or admitting them. The system ensures that patient data is properly recorded and maintained.

Task:
Construct a Use Case Diagram that identifies the main actor and all associated use cases. Clearly represent dependencies and relationships between different actions.

6. Car Rental System:
A car rental company provides a system where customers can search available cars, make reservations, rent vehicles, and return them. Customers may also make payments for rentals.

Employees manage daily operations such as updating car availability and processing rentals, while managers oversee the system and generate reports. An external insurance company interacts with the system to handle insurance-related services during rentals.

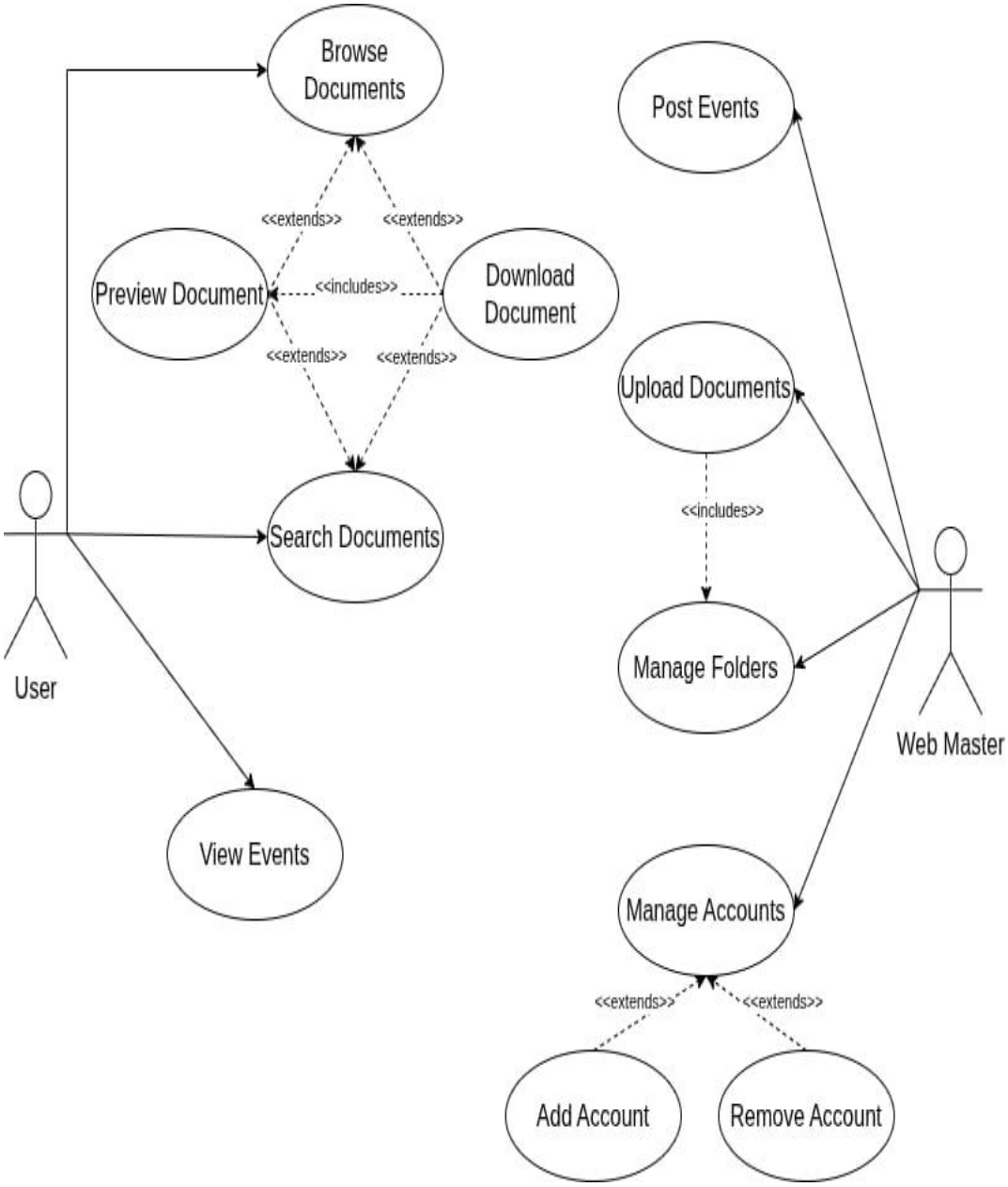
Task:
Design a Use Case Diagram including all actors (customers, employees, managers, and external organizations). Show how different actors interact with the system and represent relationships between use cases.

7. Student Registration System:
A university provides a student registration system where students can register for courses, drop courses, and view their schedules. Professors can manage course offerings and view enrolled students. Administrators oversee the system by creating courses, assigning professors, and managing student records.

Some processes depend on others, such as course creation before student registration. The system enforces rules such as prerequisites and enrollment limits.

Task:
Create a Use Case Diagram identifying all actors and their interactions. Clearly model relationships between use cases and constraints where relevant.


Enter Pin
Display Access Denied
<<includes>>
<<extends>>
Authenticate
<<includes>>
Verify Pin
Customer
Check Balance
Bank System
Withdraw Cash
Deposit Funds
Transfer Funds
Maintenance
Technician
Repair


Browse Documents
Post Events
<<extends>>
<<extends>>
Preview Document
<<includes>>
Download Document
<<extends>>
<<extends>>
Upload Documents
<<includes>>
Search Documents
User
Manage Folders
Web Master
View Events
Manage Accounts
<<extends>>
<<extends>>
Add Account
Remove Account

Exercise 05:

1. Context data flow diagram: definition and example with explanation

When it comes to simple data flow diagram examples, context one has the top place.

Context data flow diagram (also called Level 0 diagram) uses only one process to represent the functions of the entire system.

It does not go into details as marking all the processes.

The purpose is to express the system scope at a high level as well as to prevent users from deep down into complex details.

The major advantage of context DFD is simplicity.

Key context DFD characteristics:

- Simple to draw.
- No need of technical knowledge to understand it.
- Shows the system boundaries.

Steps for creating a context DFD:

- **Step1:** Define the process.
- **Step2: Create a list of all external entities (all people and systems).**
- **Step3:** Create a list of the data flows.
- **Step4:** Draw the diagram.

Let's illustrate the things with a context data flow diagram example.
Below is shown a simple context DFD drawn for a Clothes Ordering System and explanation.

Context Data Flow Diagram Example

Now, let's explain how we create the diagram.

Srep1: Define the process.
As it is a context data flow diagram, **the process is only one**. In our case, it is *Clothes Ordering System*. Draw a rectangle for the process.
Step 2: Create the list of all external entities.
In our example, the external entities are: *Customer, Clothes Store, Clothes Supplier, and the Sales Manager*. These are all entities who are involved with our system. Also, now you can draw a rectangle for each of the entities.
Step 3: Create a list of the data flows.
In between our process and the external entities, there are data flows that show a brief description of the type of information exchanged between the entities and the system.

In our example, the list of data flows includes: *Customer Order, Receipt, Clothes Order, Receipt, Clothes Order, and Management Report*.
Now, connect the rectangles with arrows signifying the data flows.

If data flows both ways between any two rectangles, create two individual arrows.

Step4: It is our diagram.

2. Level 1 data flow diagram: definition and example with explanation

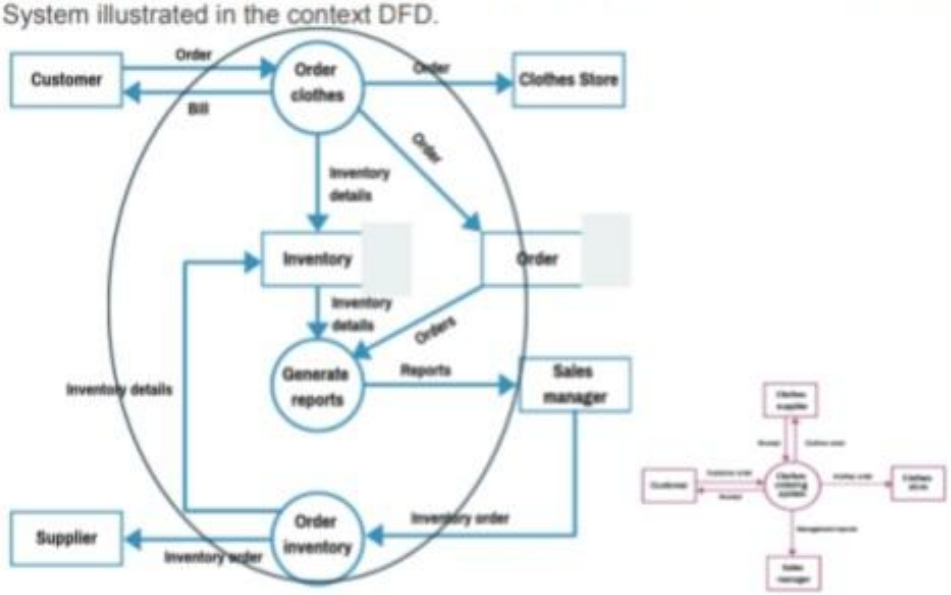
As you saw above context DFD contains only one process and **does not illustrate any data store.**
This is the main difference with level 1 DFD.

Level 1 DFD breaks down the main process into subprocesses that can then be seen on a more deep level. Also, level 1 DFD contains data stores that are used by the main process.

Steps for creating a context DFD:

- **Step1:** Define the processes (the main process and the subprocesses).
- **Step2:** Create a list of all external entities (all people and systems).
- **Step3: Create a list of the data stores.**
- **Step4:** Create a list of the data flows.
- **Step5:** Draw the diagram.

Here is our level 1 data flow example – a decomposition of the Clothes Ordering System illustrated in the context DFD.



As you see, the above Clothes Order System Data Flow Diagram Example shows three processes, four external entities, and also two data stores.

Here are the steps for creating the level 1 DFD:

- **Step 1:** Define the processes.
  The three processes are: ***Order Clothes, Generate Reports, and Order Inventory.***
- **Step 2:** Create the list of all external entities.
  The external entities are: ***Customer, Clothes Store, Sales Manager, and Supplier***
- **Step 3:** Create the list of the data stores.
  These are: ***Order and Inventory***

- **Step 4:** Create the list of the data flows
  Data flows are: ***Order, Bill, Order, Order, Inventory details, Inventory details, Orders, Reports, Inventory Order, Inventory Order, Inventory details.***

- **Step5:** Create the diagram.

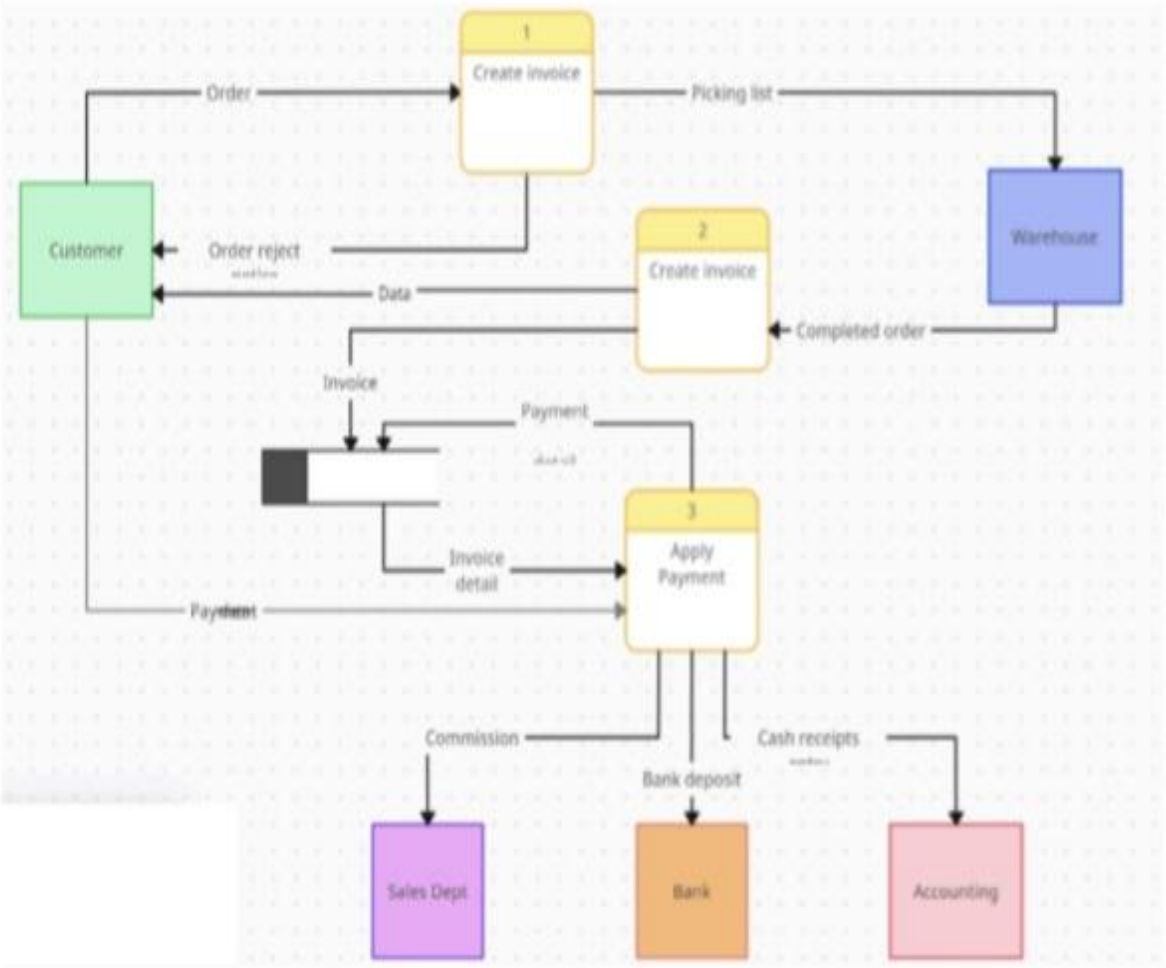
How to Create Data Flow Diagrams?

It might seem a little bit difficult to create data flow diagram examples. But in our IT world, it can be very easy and even fun to make them using the appropriate software tools.

You can use paid or free graphing software, free mind mapping software or diagramming solutions such as:

- Canva
- Lucidchard
- SmartDraw
- VisualParadigm
- Realtime Board – **this is my favorite one.**

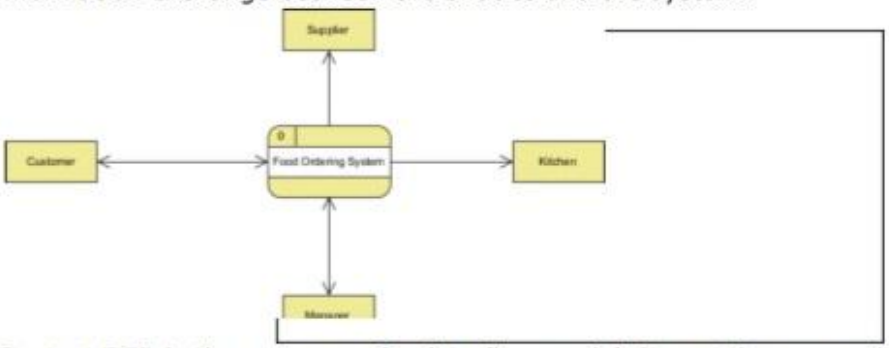
Example2:


1
Create invoice
Order
Picking list
Customer
Warehouse
Order reject
2
Create invoice
Data
Completed order
Invoice
Payment
3
Apply Payment
Invoice detail
Payment
Commission
Cash receipts
Bank deposit
Sales Dept
Bank
Accounting

Exercise 06:

Level 0 DFD Diagram

The figure below shows a context Data Flow Diagram that is drawn for a Food Ordering System. It contains a process (shape) that represents the system to model, in this case, the "Food Ordering System". It also shows the participants who will interact with the system, called the external entities. In this example, Supplier, Kitchen, Manager and Customer are the entities who will interact with the system. In between the process and the external entities, there are data flow (connectors) that indicate the existence of information exchange between the entities and the system.

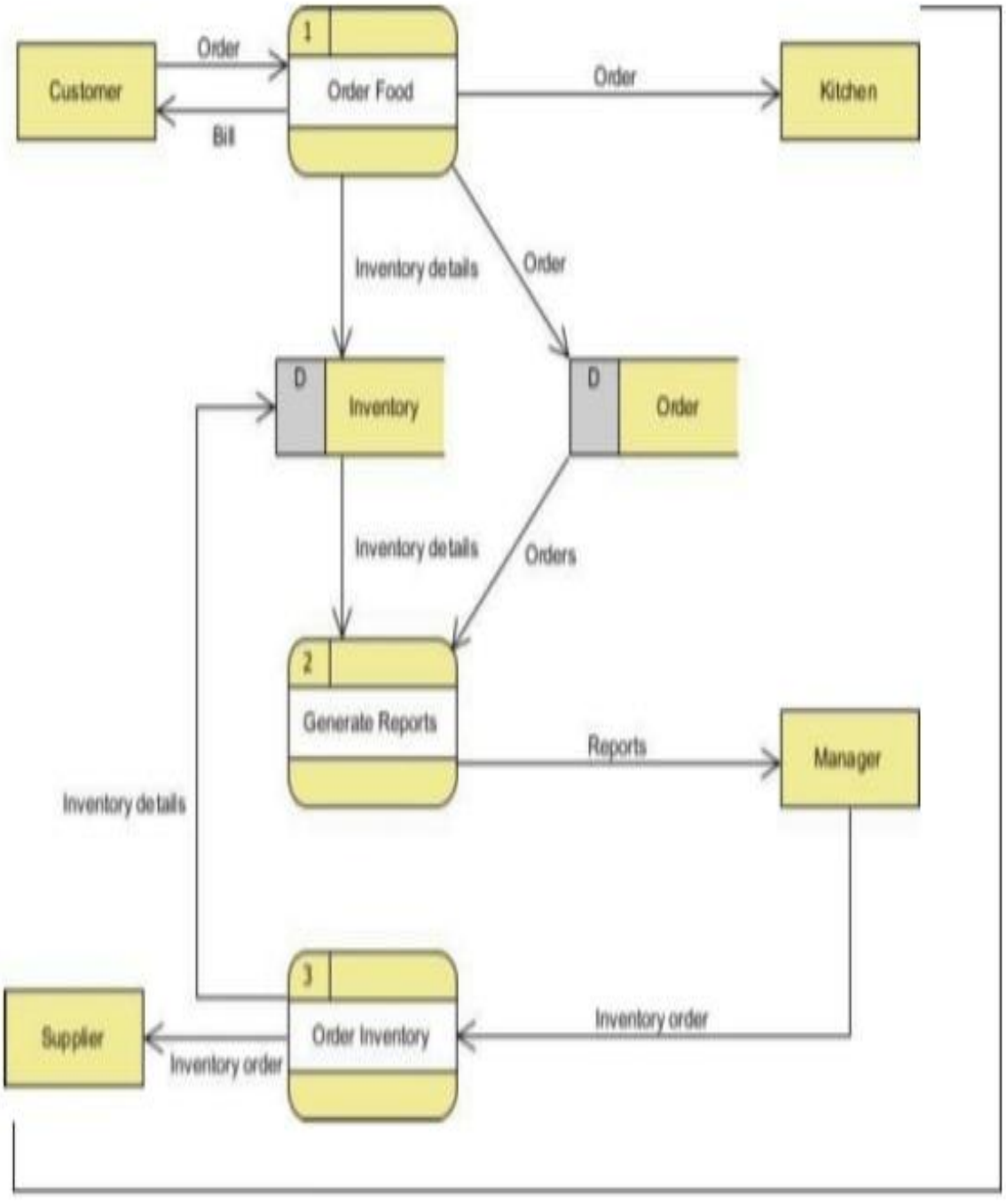


Context DFD is the entrance of a data flow model. It contains one and only one process and does not show any data store.

The Food Order System Data Flow Diagram example contains three processes, four external entities and two data stores. Draw the level 1 DFD diagram based on the following information:

- A Customer can place an order. The Order Food process receives the Order, forwards it to the Kitchen, store it in the Order data store, and store the updated Inventory details in the Inventory data store. The process also delivers a Bill to the Customer.
- Manager can receive Reports through the Generate Reports process, which takes Inventory details and Orders as input from the Inventory and Order data store respectively.
- Manager can also initiate the Order Inventory process by providing Inventory order. The process forwards the Inventory order to the Supplier and stores the updated Inventory details in the Inventory data store.

Solution:


1
Order Food
Customer
Order
Bill
Order
Kitchen
Inventory details
Order
D
Inventory
D
Order
Inventory details
Orders
2
Generate Reports
Reports
Manager
Inventory details
3
Order Inventory
Supplier
Inventory order
Inventory order